

09-Git and GitHub Revision Class

Introduction

- The session was a revision class focused on Git and GitHub, aimed at helping students learn and reinforce essential skills in version control.

Installation and Setup

- Students were instructed to uninstall previous Git installations and perform a fresh installation step-by-step in class.
- Installation covered obtaining Git from the official website (git-scm.com) for Windows, Mac, and Linux, and handling platform-specific steps.
- Importance of removing residual files and configuring the installation (choosing options like default editor Vim, selecting main as default branch) was emphasized.
- Verification of successful installation was demonstrated using git commands in the command prompt or terminal.

Understanding Git and GitHub

- Git is a distributed Version Control System (VCS) that tracks changes in files and facilitates collaboration among team members.
- GitHub is a cloud-based repository service that supports Git, enabling code sharing, backup, and teamwork.
- Git manages code locally, while GitHub provides an online platform for remote collaboration.

- Use cases and real-world scenarios highlighted how Git helps track changes, manage versions, and collaborate effectively, beyond basic file systems like Google Drive.

Version Control Systems

- Differentiation between centralized (e.g., Subversion) and distributed (e.g., Git, Mercurial) VCS was explained:
 - Centralized VCS relies on a central server; if it fails, work halts and data can be lost.
 - Distributed VCS allows each user to have full or partial project copies, improving reliability, backup, and scalability.
- Git's distributed nature is an industry standard, especially for teams needing reliability and flexibility.

Git Workflow and Key Concepts

- Explained Git's core stages:
 - **Working Directory:** Where files are created/edited.
 - **Staging Area/Index:** Buffer where changes are queued for commit (using `git add`).
 - **Local Repository:** `.git` folder where committed changes are stored (`git commit`).
 - **Remote/Cloud Repository:** (e.g., GitHub) to which commits can be pushed.
- Provided practical scenarios for staging, committing, and pushing/pulling changes.
- Emphasized manual pushing of commits to remote repositories; mentioned automation via scripting as optional.

- Discussed branch naming conventions (main vs master), reason for changing to 'main,' and how to do so.
- Described cloud integration using `git remote add origin <repo link>` and confirmation via `git remote -v`.

Basic Command-Line Operations

- Demonstrated core terminal commands for navigation and file management (`pwd`, `ls`, `cd`, `mkdir`, `touch`, `echo`, `cat`, Vim editor).
- Walked through creating project folders, initializing git repository (`git init`), creating files, adding contents, and checking Git status at every step (`git status`).
- Illustrations showed adding individual files, multiple files by type (using wildcards), and unstaging files (`git rm --cached`).
- Demonstrated file editing differences between touch, echo, and Vim for file creation/content modification.

Cloud Repository Integration

- Guided students on creating new repositories on GitHub, copying repo links, and linking local and remote repos via command line.
- Showed pushing local commits to the GitHub repository with authentication steps (using browser-based login or personal access token as required).
- Explained the significance of push and pull operations and the necessity of synchronizing with remote repositories for teamwork.
- Emphasized the importance of using command line over GUI for real-world development practices.

Troubleshooting and Q&A

- Addressed errors during installation, linking, and pushing (e.g., wrong repo links, authentication prompts, handling different options depending on the OS and GitHub's prompts).
- Clarified reasons for file visibility on GitHub after push, handling partial/unstaged file cases, branch naming issues, and protocol errors when pasting links.
- Provided practical tips for diagnosing and resolving command-line errors.

Conclusion and Further Learning

- Recap: Students should now be able to install Git, navigate basic terminal commands, set up local and cloud repositories, and perform the essential stages of version control.
- Encouraged practicing commands over GUI usage for deeper understanding and professional workflow alignment.
- Mentioned upcoming advanced classes on topics like branching and merging.
- Encouraged use of Git and GitHub for all future projects to centralize code and manage version history efficiently.